MLOps Assignment 2

Hugging Face Fine-Tuning, Experiment Tracking and Model Deployment

Student ID: G25AIT2134

Important Links

Resource	Link
Hugging Face Model	https://huggingface.co/G25AIT2134/distilbert-goodreads-genres
W&B Dashboard	https://wandb.ai/g25ait2134-iitj/mlops-assignment2/runs/5s2gs1m0
GitHub Repository	https://github.com/g25ait2134-tech/MLOpsAssignment2
Kaggle Notebook	https://www.kaggle.com/code/shyamg25ait2134/notebook828cbfd1f6

Final Results

Metric	Value
Accuracy	0.59
Weighted F1 Score	0.585
Evaluation Loss	2.297
Training Time	About 628 seconds

1. Model Selection Rationale

For this assignment, I used `distilbert/distilbert-base-uncased` from Hugging Face. I selected DistilBERT because it gives a good balance between performance and training speed. It is a smaller version of BERT created using knowledge distillation, so it keeps most of BERT's language understanding ability while being much lighter. According to the model description, DistilBERT keeps around 97% of BERT's performance while being 40% smaller and 60% faster during inference. This makes it a practical model for this assignment because the focus is not only on accuracy, but also on building an MLOps workflow that can be trained, tracked, saved, and reused.

I used the uncased version because the task is to classify Goodreads reviews into book genres. For this problem, the meaning of the words matters more than capitalization. For example, words such as "fantasy", "romance", and "mystery" are useful whether they are capitalized or not. The uncased model converts text to lowercase before tokenization, which helps reduce unnecessary differences between words like "Fantasy" and "fantasy". Since Goodreads reviews are written by users and may have inconsistent capitalization, the uncased model felt like a better choice.

DistilBERT is also suitable for sequence classification. It has 6 transformer layers, 768 hidden dimensions, and 12 attention heads. Since it was pre-trained on large English text sources such as BookCorpus and

English Wikipedia, it already has a good understanding of English language patterns. Fine-tuning it on Goodreads reviews allows the model to adapt this general understanding to the specific task of genre classification.

2. Kaggle Training Setup

I trained the model inside a Kaggle Notebook because the updated assignment required Kaggle GPU usage. In the Kaggle notebook settings, I enabled the GPU accelerator and also turned on Internet access. Internet access was important because the notebook needed to install packages, stream the Goodreads dataset from the UCSD server, connect to Weights & Biases, and push the trained model to Hugging Face Hub.

Setting	Value
Accelerator	GPU T4 x2
Internet	Enabled
Secrets Used	`WANDB_API_KEY`, `HF_TOKEN`
Framework	PyTorch and Hugging Face Transformers
Training Time	About 628 seconds

2.1 Kaggle Secrets

I did not hardcode the API keys directly in the notebook. Instead, I stored both tokens in Kaggle Secrets. The Weights & Biases key was saved as `WANDB_API_KEY`, and the Hugging Face token was saved as `HF_TOKEN`. These were then loaded in the notebook using `UserSecretsClient`.

```
from kaggle_secrets import UserSecretsClient

secrets = UserSecretsClient()
WANDB_API_KEY = secrets.get_secret("WANDB_API_KEY")
HF_TOKEN = secrets.get_secret("HF_TOKEN")

import os
os.environ["WANDB_API_KEY"] = WANDB_API_KEY
os.environ["HF_TOKEN"] = HF_TOKEN
```

This was useful because it keeps private credentials out of the notebook and GitHub repository.

2.2 Creating the API Keys

For Weights & Biases, I created an account on `wandb.ai`, opened the API key section in settings, generated a new key, and added it to Kaggle Secrets as `WANDB_API_KEY`.

For Hugging Face, I created an access token from the Hugging Face settings page. Since I needed to push the trained model to Hugging Face Hub, I used a token with write permission. I then added this token to Kaggle Secrets as `HF_TOKEN`.

2.3 Training Hyperparameters

Hyperparameter	Value
Model	`distilbert/distilbert-base-uncased`
Epochs	3

Hyperparameter	Value
Training batch size	16
Evaluation batch size	32
Learning rate	3e-5
Warmup steps	100
Weight decay	0.01
Logging steps	50
Evaluation strategy	Every epoch
Save strategy	Every epoch
Load best model at end	Yes
W&B tracking	`report_to="wandb"`

3. Experiment Tracking with W&B

Weights & Biases was used to track the training run. I connected it through the Hugging Face `Trainer` by setting `report_to="wandb"` inside `TrainingArguments`. After doing this, the trainer automatically logged training loss, evaluation loss, accuracy, F1 score, learning rate, runtime details, and GPU-related information.

I also used `wandb.init()` before training to store the main configuration values for the run. This included the model name, number of epochs, batch size, learning rate, maximum sequence length, dataset name, and platform.

```
wandb.init(
    project="mlops-assignment2",
    name="distilbert-run-1",
    config={
        "model": model_name,
        "epochs": 3,
        "batch_size": 16,
        "learning_rate": 3e-5,
        "max_length": 512,
        "dataset": "UCSD Goodreads",
        "platform": "Kaggle",
    }
)
```

After training, I explicitly logged the final metrics to W&B. I also saved the complete classification report as `eval_report.json` and uploaded it as a W&B artifact. This makes the evaluation output easier to trace later.

```
wandb.log({
    "final/loss": 2.297,
    "final/accuracy": 0.59,
    "final/f1": 0.585,
})

artifact = wandb.Artifact("eval-report", type="evaluation")
artifact.add_file("eval_report.json")
wandb.log_artifact(artifact)
```

W&B Metric	Value
Final accuracy	0.59

W&B Metric	Value
Final weighted F1	0.585
Final eval loss	2.297
Final train loss	1.712
Train samples/sec	37.5
Runtime	628 seconds
Global steps	600

4. Evaluation Results

The final model was evaluated on a test set of 1,600 Goodreads reviews, with 200 reviews from each genre. The model achieved an accuracy of 59% and a weighted F1 score of 0.585. Since this is an 8-class classification problem, random guessing would be around 12.5%, so the result shows that the model learned useful patterns from the reviews.

Genre	Precision	Recall	F1 Score
children	0.632	0.585	0.608
comics & graphic	0.761	0.810	0.785
fantasy & paranormal	0.430	0.490	0.458
history & biography	0.598	0.550	0.573
mystery / thriller	0.490	0.605	0.541
poetry	0.741	0.815	0.776
romance	0.613	0.585	0.598
young adult	0.424	0.280	0.337
weighted average	0.586	0.590	0.585

The strongest results were for Poetry and Comics & Graphic. This makes sense because these genres often have very specific words in their reviews. Poetry reviews may mention words like "verse", "poem", or "lyrical", while comics reviews often mention "art", "panels", or "illustrations". These words make the genres easier to identify.

The weaker results were for Young Adult and Fantasy & Paranormal. These categories overlap a lot. Many young adult books include fantasy, romance, mystery, and adventure elements, so the model can confuse them with other genres. Fantasy and young adult books also share themes like magic, coming-of-age, and adventure. This likely explains why those classes had lower F1 scores.

Overall, the model performance is reasonable for a small 3-epoch fine-tuning run with 1,000 reviews per genre.

5. Model Publishing on Hugging Face Hub

After training and evaluation, I pushed both the trained model and tokenizer to Hugging Face Hub from the Kaggle notebook. I first logged in using the Hugging Face token stored in Kaggle Secrets.

```
from huggingface_hub import login
```

```
login(token=HF_TOKEN)

repo_name = "G25AIT2134/distilbert-goodreads-genres"

model.push_to_hub(repo_name)
tokenizer.push_to_hub(repo_name)

wandb.run.summary["huggingface_model"] = f"https://huggingface.co/{repo_name}"
```

The final model is available at:

https://huggingface.co/G25AIT2134/distilbert-goodreads-genres

It can be loaded using the Hugging Face `pipeline` API:

```
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="G25AIT2134/distilbert-goodreads-genres"
)

classifier("A gripping tale of dragons and ancient kingdoms...")
```

The GitHub repository also includes an `inference.py` file, which shows how to load the model and run a sample prediction.

6. Challenges and Learnings

One challenge was getting the dataset to load properly in Kaggle. The Goodreads data is stored as compressed JSON files and is streamed through HTTP. This only worked after enabling Internet in the Kaggle session settings. Without that setting, the dataset download failed.

Another issue was label ordering. Initially, using Python's `set()` for labels can create a different label order each time the notebook runs. This can make results harder to reproduce. A better approach is to use `sorted(set(train_labels))`, so that the label-to-ID mapping stays consistent across runs.

I also learned that the position of `wandb.finish()` matters. If it is called too early, later logging steps, such as adding the Hugging Face model URL to the W&B summary, may not work properly. Moving `wandb.finish()` to the final cell fixed this.

The choice between cased and uncased DistilBERT was also an important learning point. The original notebook used a cased model, but for this genre classification task, capitalization did not seem very important. Since the reviews are user-written and inconsistent, the uncased model was more suitable.

If I were to improve this project further, I would train on a larger sample of reviews per genre. I used 1,000 reviews per genre, but using 5,000 or more could improve performance. I would also create a separate validation set instead of only train and test sets. This would make hyperparameter tuning more reliable. Another improvement would be running multiple W&B experiments with different learning rates and batch sizes to compare results more clearly. Finally, I would add a proper model card on Hugging Face explaining the dataset, intended use, limitations, and evaluation results.

The main learning from this assignment was that MLOps is not only about training a model. It is also about making the training process reproducible, tracking experiments properly, storing credentials safely, evaluating the model clearly, and publishing the final model so that others can use it.